Comprehensive SQL Learning Guide

Beginner to Advanced

Table of Contents

1. Introduction to Databases & SQL
2. SQL Basics
3. Data Definition Language (DDL)
4. Data Manipulation Language (DML)
5. Data Query Language (SELECT)
6. Filtering & Conditions
7. Aggregations & Grouping
8. Joins & Relationships
9. Subqueries & CTEs
10. Constraints & Keys
11. Indexes & Performance
12. Views, Procedures & Functions
13. Transactions & ACID Properties
14. SQL for Analytics & Reporting
15. SQL Interview Questions & Scenarios

Chapter 1: Introduction to Databases & SQL

What is a Database?

A **database** is an organized collection of structured data stored and accessed electronically. Think of it like an advanced Excel spreadsheet that can:

- Store much larger amounts of data
- Provide faster access to specific information
- Ensure data safety and integrity
- Allow multiple users to work simultaneously
- Support complex data relationships

Example Database Structure:

- Database: company_db
  - Tables: employees, departments, salaries, projects
  - Each table contains related data

What is a Table?

A **table** is like an Excel sheet with rows and columns. Each table stores one type of entity.

Example: employees table

id	name	department	salary
1	Rahul	IT	50000
2	Neha	HR	45000
3	Amit	IT	55000

Terminology:

- **Row** = Record (one employee's data)
- **Column** = Attribute (a field like name, department)
- **Cell** = Individual data value

What is SQL?

SQL (Structured Query Language) is a standardized language used to communicate with databases. It allows you to:

- **Store data** (CREATE, INSERT)
- **Read data** (SELECT)
- **Update data** (UPDATE)
- **Delete data** (DELETE)
- **Manage structure** (ALTER, DROP)

SQL is the universal language for working with relational databases. Whether you use PostgreSQL, MySQL, SQL Server, or Oracle, the core SQL syntax remains the same.

Types of Databases

Relational Databases (RDBMS)

Uses structured tables with predefined schemas. Data is organized in rows and columns with relationships between tables.

Examples: PostgreSQL, MySQL, SQL Server, Oracle, SQLite

Best for: Business applications, financial systems, structured data

NoSQL Databases

Non-relational databases designed for flexible, unstructured data. Store data as documents, key-value pairs, graphs, or wide columns.

Examples: MongoDB, Cassandra, Redis, DynamoDB

Best for: Real-time analytics, content management, big data, flexible schemas

How SQL is Used in Real Projects

In Data Engineering:

- Building ETL pipelines to extract, transform, and load data
- Creating data warehouse schemas
- Running batch queries for data quality checks
- Implementing real-time data processing

In Business Intelligence:

- Querying data for reports and dashboards
- Creating fact and dimension tables
- Aggregating data for analysis
- Supporting BI tools like Power BI

In Web Development:

- Storing user data, products, and transactions
- Managing user authentication
- Tracking orders and inventory
- Generating analytics

In Analytics:

- Running complex queries for insights
- Creating summary statistics
- Building predictive models
- Detecting trends and patterns

SQL Mental Model

Think of SQL as asking questions to your database.

Instead of thinking "How do I write code?", think "What question am I asking the data?"

Examples:

- "Give me all employees"
- "Show me employees from the IT department"
- "Find employees with salary > 50,000"
- "Count how many employees work in each department"
- "Show me the top 5 highest-paid employees"

Once you ask the question clearly, writing the SQL is straightforward.

Chapter 2: SQL Basics

SQL Syntax Rules

SQL has consistent rules that help you write correct queries:

1. **SQL is case-insensitive** for keywords (SELECT, FROM, WHERE are same as select, from, where)
2. **Whitespace doesn't matter** (you can format queries however you like)
3. **End statements with semicolon** (;) to mark the end
4. **String values use single quotes** ('value') not double quotes
5. **Statements are read left-to-right, top-to-bottom**

Keywords, Identifiers, and Data Types

Keywords

Keywords are reserved words with special meaning in SQL. Examples: SELECT, FROM, WHERE, JOIN, GROUP BY

Identifiers

Identifiers are names you give to tables, columns, and other objects. Rules:

- Start with a letter or underscore
- Can contain letters, numbers, underscores
- Case-sensitive in most databases
- Use lowercase with underscores (snake_case) by convention: employee_id, first_name

Common Data Types

Data Type	Description	Example
INT	Integer numbers	42, -5, 1000
FLOAT / DECIMAL	Decimal numbers	3.14, 99.99
VARCHAR(n)	Text of variable length	'Rahul', 'New Delhi'
CHAR(n)	Text of fixed length	'A', 'NY'
DATE	Calendar date	2026-01-18
TIMESTAMP	Date and time	2026-01-18 09:30:00
BOOLEAN	True/False	TRUE, FALSE
NULL	Missing value	NULL

Comments in SQL

Comments help explain your code. They are ignored during execution.

Single-line comment:
SELECT name FROM employees; -- Get all employee names

Multi-line comment:
/* This is a multi-line comment
that spans multiple lines
Use it for detailed explanations */
SELECT * FROM employees;

Core SQL Commands Overview

These 8 commands cover 90% of SQL usage:

Command	Purpose	Example
SELECT	Read data	SELECT name FROM employees
INSERT	Add new data	INSERT INTO employees VALUES (...)
UPDATE	Modify existing data	UPDATE employees SET salary = 60000
DELETE	Remove data	DELETE FROM employees WHERE id = 1
WHERE	Filter rows	WHERE salary > 50000
ORDER BY	Sort results	ORDER BY salary DESC
GROUP BY	Aggregate data	GROUP BY department
JOIN	Combine tables	JOIN departments ON ...

Chapter 3: Data Definition Language (DDL)

CREATE Statement

The CREATE statement is used to create new database objects (tables, databases, views, indexes).

Create a New Table:

```
CREATE TABLE employees (
id INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
department VARCHAR(50),
salary INT,
hire_date DATE
);
```

Key constraints:

- **PRIMARY KEY** - Uniquely identifies each row
- **NOT NULL** - Column must have a value
- **UNIQUE** - All values in column must be different
- **DEFAULT** - Set a default value if none provided

Create a New Database:

```
CREATE DATABASE company_db;
```

ALTER Statement

Modify an existing table structure (add, modify, or delete columns).

Add a New Column:

```
ALTER TABLE employees ADD COLUMN email VARCHAR(100);
```

Modify an Existing Column:

```
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(10,2);
```

Drop a Column:

```
ALTER TABLE employees DROP COLUMN email;
```

Rename a Column:

```
ALTER TABLE employees RENAME COLUMN hire_date TO start_date;
```

DROP Statement

Completely remove a table or database (be careful - this is permanent!).

Drop a Table:

```
DROP TABLE employees;
```

Drop a Database:

```
DROP DATABASE company_db;
```

TRUNCATE Statement

Remove all rows from a table but keep the table structure intact. Faster than DELETE for removing all rows.

Truncate a Table:

```
TRUNCATE TABLE employees;
```

Key Difference:

- TRUNCATE: Removes all rows quickly, frees space
- DELETE: Can have WHERE clause, slower, keeps transaction log

Table Design Best Practices

1. Choose Appropriate Data Types

```
-- GOOD: Right data type for each column
CREATE TABLE employees (
id INT,
name VARCHAR(100),
hire_date DATE,
salary DECIMAL(10,2)
);

-- BAD: Using VARCHAR for everything
CREATE TABLE employees (
id VARCHAR(10),
name VARCHAR(100),
hire_date VARCHAR(20),
salary VARCHAR(20)
);
```

2. Use Primary Keys

```
-- Every table should have a unique identifier
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
name VARCHAR(100),
department_id INT
);
```

3. Use Meaningful Column Names

```
-- GOOD: Clear, descriptive names
CREATE TABLE employees (
employee_id INT,
first_name VARCHAR(50),
last_name VARCHAR(50),
```

```
employee_salary DECIMAL(10,2)
);

-- BAD: Unclear names
CREATE TABLE emp (
eid INT,
fn VARCHAR(50),
ln VARCHAR(50),
sal DECIMAL(10,2)
);
```

4. Normalize Your Data

Organize data to minimize redundancy and improve consistency. Related data should be in separate tables with relationships.

```
-- GOOD: Separate tables for departments and employees
CREATE TABLE departments (
department_id INT PRIMARY KEY,
department_name VARCHAR(100)
);

CREATE TABLE employees (
employee_id INT PRIMARY KEY,
name VARCHAR(100),
department_id INT FOREIGN KEY REFERENCES departments(department_id)
);

-- BAD: Repeating department info in employees table
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
name VARCHAR(100),
department_name VARCHAR(100),
department_office VARCHAR(100),
department_budget INT
);
```

5. Set Constraints Appropriately

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
email VARCHAR(100) UNIQUE, -- Emails must be unique
name VARCHAR(100) NOT NULL, -- Name is required
status VARCHAR(20) DEFAULT 'Active', -- Set default status
salary DECIMAL(10,2) CHECK (salary > 0) -- Salary must be positive
);
```

Chapter 4: Data Manipulation Language (DML)

INSERT Statement

Add new rows to a table.

Insert Single Row (All Columns):

```
INSERT INTO employees VALUES (1, 'Rahul', 'IT', 50000);
```

Insert Single Row (Specific Columns):

```
INSERT INTO employees (id, name, department)
VALUES (2, 'Neha', 'HR');
```

Insert Multiple Rows:

```
INSERT INTO employees (id, name, department, salary)
VALUES
(3, 'Amit', 'IT', 55000),
(4, 'Priya', 'Finance', 52000),
(5, 'Deepak', 'HR', 48000);
```

Insert from Another Table:

```
INSERT INTO employees_backup
SELECT * FROM employees WHERE department = 'IT';
```

UPDATE Statement

Modify existing data in a table.

Update Single Column:

```
UPDATE employees
SET salary = 60000
WHERE id = 1;
```

Update Multiple Columns:

```
UPDATE employees
SET salary = 60000, department = 'Manager'
WHERE id = 1;
```

Update with Calculation:

```
UPDATE employees
SET salary = salary * 1.10 -- 10% raise
WHERE department = 'IT';
```

⚠ **IMPORTANT:** Always include a WHERE clause to avoid updating all rows!

DELETE Statement

Remove rows from a table.

Delete Specific Rows:

```
DELETE FROM employees
WHERE id = 1;
```

Delete with Multiple Conditions:

```
DELETE FROM employees
WHERE department = 'IT' AND salary < 50000;
```

⚠ **CRITICAL:** Always use WHERE clause. Running `DELETE FROM employees;` deletes everything!

MERGE Statement

Combines INSERT, UPDATE, and DELETE in a single statement. Useful for synchronizing data.

Merge Syntax:

```
MERGE INTO target_table AS t
USING source_table AS s
ON t.id = s.id
WHEN MATCHED THEN
UPDATE SET t.name = s.name, t.salary = s.salary
WHEN NOT MATCHED THEN
INSERT (id, name, salary) VALUES (s.id, s.name, s.salary);
```

Use Case: Syncing employee data from a source system to your database.

Transaction Basics

A **transaction** is a sequence of SQL statements treated as one atomic unit. Either all statements succeed or all roll back.

Key Commands:

- **COMMIT** - Save changes permanently
- **ROLLBACK** - Undo changes
- **SAVEPOINT** - Create a recovery point

Example Transaction:

```
BEGIN; -- Start transaction

UPDATE accounts SET balance = balance - 1000
WHERE account_id = 101;

UPDATE accounts SET balance = balance + 1000
WHERE account_id = 102;
```

```
COMMIT; -- Save both changes or rollback both
```

Example with Rollback:

```
BEGIN;

DELETE FROM employees WHERE id = 1;

-- Oops, wrong employee! Undo the deletion
ROLLBACK;
```

Chapter 5: Data Query Language (SELECT)

SELECT Syntax

The SELECT statement is the most important SQL command. It retrieves data from the database.

Basic Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition
GROUP BY column
HAVING condition
ORDER BY column
LIMIT n;
```

Order of Execution (Important):

1. FROM - Identify the table
2. JOIN - Combine tables (if used)
3. WHERE - Filter rows
4. GROUP BY - Group rows
5. HAVING - Filter groups
6. SELECT - Choose columns
7. ORDER BY - Sort results
8. LIMIT - Restrict output

Basic SELECT Examples

Select All Columns:

```
SELECT * FROM employees;
```

Output: All columns and all rows from the employees table.

Select Specific Columns:

```
SELECT name, salary FROM employees;
```

Output: Only the name and salary columns.

Select with Expression:

```
SELECT name, salary * 12 AS annual_salary
FROM employees;
```

Output: Employee names and their annual salary calculated from monthly salary.

Aliases

Column Aliases - Give a new name to a column in output:

```
SELECT
name AS employee_name,
salary AS monthly_salary,
salary * 12 AS annual_salary
FROM employees;
```

Table Aliases - Shorten table names (useful in joins):

```
SELECT e.name, e.salary
FROM employees AS e;
```

Or shorter:

```
SELECT e.name, e.salary
FROM employees e;
```

DISTINCT Keyword

Remove duplicate rows from results.

Example Data (without DISTINCT):

```
SELECT department FROM employees;
```

Result:
department
IT
HR
IT
Finance
HR
IT

With DISTINCT:

```
SELECT DISTINCT department FROM employees;
```

Result:
department
IT
HR
Finance

ORDER BY Clause

Sort results in ascending (ASC) or descending (DESC) order.

Sort by Single Column (Ascending - default):

```
SELECT name, salary
FROM employees
ORDER BY salary;
```

Sort Descending:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Sort by Multiple Columns:

```
SELECT name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

This sorts first by department, then within each department by salary highest to lowest.

LIMIT and TOP

Restrict the number of rows returned.

LIMIT (PostgreSQL, MySQL, SQLite):

```
SELECT * FROM employees LIMIT 5;
```

Returns the first 5 rows.

TOP (SQL Server):

```
SELECT TOP 5 * FROM employees;
```

OFFSET - Skip Rows (Pagination):

```
SELECT * FROM employees
ORDER BY salary DESC
LIMIT 5 OFFSET 10;
```

Returns 5 rows starting from row 11 (skips first 10).

Practical Example: Find Top 3 Highest-Paid Employees:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Chapter 6: Filtering & Conditions

WHERE Clause

Filter rows based on conditions.

Basic Syntax:

SELECT * FROM table_name WHERE condition;

Comparison Operators

Operator	Meaning
=	Equal to
!= or <>	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Examples:

```
-- Salary greater than 50,000
SELECT * FROM employees WHERE salary > 50000;

-- Department equals IT
SELECT * FROM employees WHERE department = 'IT';

-- Salary less than or equal to 45,000
SELECT * FROM employees WHERE salary <= 45000;
```

AND Operator

Both conditions must be true.

```
SELECT * FROM employees
WHERE department = 'IT' AND salary > 50000;
```

Returns only IT employees earning more than 50,000.

OR Operator

At least one condition must be true.

```
SELECT * FROM employees
WHERE department = 'IT' OR department = 'HR';
```

Returns employees from either IT or HR department.

NOT Operator

Negates a condition.

```
SELECT * FROM employees
WHERE NOT department = 'IT';
```

Returns all employees NOT in the IT department.

IN Operator

Check if value exists in a list.

```
SELECT * FROM employees
WHERE department IN ('IT', 'HR', 'Finance');
```

Same as:

```
SELECT * FROM employees
WHERE department = 'IT'
OR department = 'HR'
OR department = 'Finance';
```

BETWEEN Operator

Check if value is within a range (inclusive).

```
-- Salary between 40,000 and 60,000
SELECT * FROM employees
WHERE salary BETWEEN 40000 AND 60000;
```

Also works with dates:

```
SELECT * FROM employees
WHERE hire_date BETWEEN '2020-01-01' AND '2023-12-31';
```

LIKE Operator

Search for patterns in text. Use wildcards:

- `%` = any number of characters
- `_` = exactly one character

Examples:

```
-- Names starting with 'R'
SELECT * FROM employees WHERE name LIKE 'R%';

-- Names ending with 'il'
SELECT * FROM employees WHERE name LIKE '%il';

-- Names containing 'ah'
SELECT * FROM employees WHERE name LIKE '%ah%';

-- Names with exactly 4 characters
SELECT * FROM employees WHERE name LIKE '____';
```

NULL Handling

NULL represents missing or unknown values.

Check if NULL:

```
SELECT * FROM employees WHERE email IS NULL;
```

Check if NOT NULL:

```
SELECT * FROM employees WHERE email IS NOT NULL;
```

⚠ **IMPORTANT:** Cannot use `= NULL` or `!= NULL`. Must use IS NULL or IS NOT NULL.

```
-- WRONG - will not work
SELECT * FROM employees WHERE email = NULL;

-- CORRECT
SELECT * FROM employees WHERE email IS NULL;
```

Combining Multiple Conditions

Use parentheses to control logic:

```
SELECT * FROM employees
WHERE (department = 'IT' OR department = 'HR')
AND salary > 50000;
```

This finds IT or HR employees earning more than 50,000.

Without parentheses, the logic might be evaluated differently due to operator precedence.

Chapter 7: Aggregations & Grouping

Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single result.

COUNT

Count the number of rows or non-NULL values.

```
-- Count all rows
SELECT COUNT(*) FROM employees;

-- Count non-NULL values in a column
SELECT COUNT(email) FROM employees;

-- Count with condition
SELECT COUNT(*) FROM employees WHERE department = 'IT';
```

SUM

Add up numeric values.

```
-- Total salary expense
SELECT SUM(salary) FROM employees;

-- Total salary for IT department
SELECT SUM(salary) FROM employees WHERE department = 'IT';
```

AVG

Calculate the average of numeric values.

```
-- Average salary
SELECT AVG(salary) FROM employees;

-- Average salary by department (needs GROUP BY)
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

MIN and MAX

Find minimum and maximum values.

```
-- Lowest and highest salary
SELECT MIN(salary), MAX(salary) FROM employees;

-- Min and max salary by department
SELECT department, MIN(salary), MAX(salary)
FROM employees
GROUP BY department;
```

GROUP BY Clause

Group rows by one or more columns and apply aggregate functions.

Basic Syntax:

```
SELECT column1, COUNT(*)
FROM table_name
GROUP BY column1;
```

Key Rule (The Golden Rule):

If your SELECT contains any non-aggregated column, that column MUST appear in GROUP BY.

Examples:

```
-- Count employees by department
SELECT department, COUNT(*) AS count
FROM employees
GROUP BY department;
```

Result:
department | count
IT | 3
HR | 2
Finance | 2

Multiple Columns in GROUP BY:

```
-- Count employees by department and status
SELECT department, status, COUNT(*)
FROM employees
GROUP BY department, status;
```

HAVING Clause

Filter groups after GROUP BY (similar to WHERE but for groups).

⚠ Important Distinction:

- **WHERE** filters individual rows BEFORE grouping
- **HAVING** filters groups AFTER grouping

```
-- Find departments with more than 2 employees
SELECT department, COUNT() AS count
FROM employees
GROUP BY department
HAVING COUNT() > 2;
```

With Conditions on Aggregates:

```
-- Find departments with average salary > 50,000
SELECT department, AVG(salary)
```

```
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

Complete Example: Sales Analysis

```
SELECT
department,
COUNT() AS employee_count,
AVG(salary) AS avg_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary,
SUM(salary) AS total_salary
FROM employees
WHERE hire_date >= '2020-01-01'
GROUP BY department
HAVING COUNT() >= 2
ORDER BY avg_salary DESC;
```

This query:

1. Filters employees hired after 2020
2. Groups by department
3. Calculates statistics for each group
4. Filters groups with at least 2 employees
5. Sorts by average salary (highest first)

Common Mistakes

Mistake 1: Non-aggregated column without GROUP BY

```
-- WRONG
SELECT department, name, AVG(salary)
FROM employees
GROUP BY department;

-- Which employee's name should it show? Error!

-- CORRECT
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Mistake 2: Using WHERE instead of HAVING

```
-- WRONG - WHERE filters before grouping
SELECT department, COUNT()
FROM employees
WHERE COUNT() > 2 -- Error! Can't use aggregate in WHERE
GROUP BY department;

-- CORRECT - HAVING filters after grouping
SELECT department, COUNT()
```

FROM employees
GROUP BY department
HAVING COUNT() > 2;

Mistake 3: Forgetting GROUP BY

-- WRONG - mixing aggregated and non-aggregated without GROUP BY
SELECT name, COUNT(*)
FROM employees;

-- CORRECT - if you want all employees listed
SELECT name, COUNT(*) OVER ()
FROM employees;

-- Or GROUP BY each name
SELECT name, COUNT(*)
FROM employees
GROUP BY name;

Chapter 8: Joins & Relationships

Understanding Table Relationships

Tables are related through **foreign keys** - columns that reference another table's primary key.

Example:

employees table:

- employee_id (Primary Key)
- name
- department_id (Foreign Key → references departments)

departments table:

- department_id (Primary Key)
- department_name

INNER JOIN

Returns rows that have matches in BOTH tables.

Syntax:

SELECT columns
FROM table1
INNER JOIN table2
ON table1.foreign_key = table2.primary_key;

Example:

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Returns only employees that have a matching department.

LEFT JOIN

Returns ALL rows from the left table, plus matching rows from the right table. Non-matching right table rows show NULL.

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.foreign_key = table2.primary_key;
```

Example:

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

Returns all employees, with department names if available (NULL if no match).

RIGHT JOIN

Opposite of LEFT JOIN. Returns ALL rows from the right table, plus matching rows from the left table.

Example:

```
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

Returns all departments, with employee names if they exist.

FULL JOIN

Returns ALL rows from both tables. Unmatched rows show NULL.

Example:

```
SELECT e.name, d.department_name
FROM employees e
FULL JOIN departments d
ON e.department_id = d.department_id;
```

Returns all employees AND all departments, matching where possible.

SELF JOIN

Join a table to itself. Useful for hierarchical data.

Example: Find employees and their managers (both are in employees table)

```
SELECT
e.name AS employee,
m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.employee_id;
```

Join Use Cases

Join with WHERE:

```
SELECT e.name, d.department_name, e.salary
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id
WHERE e.salary > 50000;
```

Join with Aggregation:

```
SELECT d.department_name, COUNT(e.employee_id) AS count
FROM departments d
LEFT JOIN employees e
ON d.department_id = e.department_id
GROUP BY d.department_name;
```

Multiple Joins:

```
SELECT
e.name,
d.department_name,
p.project_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id
INNER JOIN projects p ON e.employee_id = p.employee_id;
```

Chapter 9: Subqueries & CTEs

Subqueries

A **subquery** (inner query) is a query inside another query.

Types:

- **Non-correlated** - Runs once, independent of outer query
- **Correlated** - References columns from outer query, runs for each row

Non-Correlated Subqueries

Independent inner query that runs once.

Example 1: Find employees with above-average salary

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

The subquery `(SELECT AVG(salary) FROM employees)` runs first, returns average, then used in WHERE.

Example 2: Find employees in departments with > 2 employees

```
SELECT name, department_id
FROM employees
WHERE department_id IN
(SELECT department_id
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 2);
```

Correlated Subqueries

Inner query references columns from outer query. Runs for each outer row (slower).

Example: Find employees who earn more than their department average

```
SELECT name, salary, department_id
FROM employees e1
WHERE salary >
(SELECT AVG(salary)
FROM employees e2
WHERE e2.department_id = e1.department_id);
```

For each employee (e1), the subquery calculates that department's average and compares.

CTEs (Common Table Expressions) - WITH Clause

A **CTE** is a named temporary result set used within a SELECT, INSERT, UPDATE, or DELETE.

Syntax:

```
WITH cte_name AS (
SELECT columns FROM table WHERE condition
)
SELECT * FROM cte_name;
```

Example 1: Simple CTE

```
WITH high_earners AS (
SELECT name, salary
```

```
FROM employees
WHERE salary > 60000
)
SELECT name, salary
FROM high_earners
ORDER BY salary DESC;
```

Example 2: Department Averages CTE

```
WITH dept_avg AS (
SELECT department_id, AVG(salary) AS avg_sal
FROM employees
GROUP BY department_id
)
SELECT e.name, e.salary, da.avg_sal
FROM employees e
JOIN dept_avg da ON e.department_id = da.department_id
WHERE e.salary > da.avg_sal;
```

Example 3: Multiple CTEs

```
WITH sales_2023 AS (
SELECT * FROM sales WHERE year = 2023
),
top_products AS (
SELECT product_id, SUM(amount) as total
FROM sales_2023
GROUP BY product_id
ORDER BY total DESC
LIMIT 10
)
SELECT * FROM top_products;
```

Recursive CTEs

A CTE that references itself. Useful for hierarchical data.

Example: Employee hierarchy (manager relationships)

```
WITH RECURSIVE emp_hierarchy AS (
-- Base case: top-level employees (no manager)
SELECT employee_id, name, manager_id, 1 as level
FROM employees
WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case: employees reporting to previous level
    SELECT e.employee_id, e.name, e.manager_id, eh.level + 1
```

```
    FROM employees e
    INNER JOIN emp_hierarchy eh ON e.manager_id = eh.employee_id
)
SELECT * FROM emp_hierarchy
ORDER BY level, name;
```

Subquery vs CTE vs Join

When to use each:

Method	Best For	Pros	Cons
Subquery	Single use logic	Inline, simple	Hard to read if complex
CTE	Multi-use or complex logic	Readable, reusable	Slightly more code
JOIN	Combining related data	Fast, set-based	Need common key

Rule of thumb:

- Simple one-time logic → Subquery
- Complex or reused logic → CTE
- Combining data from tables → JOIN

Chapter 10: Constraints & Keys

PRIMARY KEY

Uniquely identifies each row in a table. Every table should have one.

Properties:

- Must be unique
- Cannot be NULL
- Only one per table
- Enforces data integrity

Creating a Primary Key:

```
-- Method 1: Column constraint
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
name VARCHAR(100)
);
```

```
-- Method 2: Table constraint
CREATE TABLE employees (
employee_id INT,
name VARCHAR(100),
PRIMARY KEY (employee_id)
);

-- Method 3: Composite primary key
CREATE TABLE sales (
product_id INT,
date DATE,
amount DECIMAL,
PRIMARY KEY (product_id, date)
);
```

FOREIGN KEY

Establishes relationships between tables. A column that references another table's primary key.

Example:

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
name VARCHAR(100),
department_id INT,
FOREIGN KEY (department_id) REFERENCES departments(department_id)
);
```

Benefits:

- Enforces referential integrity
- Prevents orphaned records
- Maintains data consistency

With CASCADE:

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
department_id INT,
FOREIGN KEY (department_id) REFERENCES departments(department_id)
ON DELETE CASCADE -- Delete employee if department deleted
);
```

UNIQUE Constraint

Ensures all values in a column are unique (but allows NULL).

Example:

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
email VARCHAR(100) UNIQUE,
```

```
name VARCHAR(100)
);
```

Difference from PRIMARY KEY:

- Can have multiple UNIQUE constraints per table
- Can allow NULL values
- PRIMARY KEY does not allow NULL

CHECK Constraint

Ensures column values meet a condition.

Example:

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
name VARCHAR(100),
salary DECIMAL(10,2) CHECK (salary > 0),
age INT CHECK (age >= 18 AND age <= 70),
status VARCHAR(20) CHECK (status IN ('Active', 'Inactive', 'On Leave'))
);
```

DEFAULT Constraint

Sets a default value if no value is provided.

Example:

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
name VARCHAR(100),
status VARCHAR(20) DEFAULT 'Active',
hire_date DATE DEFAULT CURRENT_DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

When inserting a row without specifying these columns, they get the default values.

Complete Table Example with All Constraints

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY AUTO_INCREMENT,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
salary DECIMAL(10,2) NOT NULL CHECK (salary >= 0),
department_id INT NOT NULL,
status VARCHAR(20) DEFAULT 'Active',
hire_date DATE DEFAULT CURRENT_DATE,
FOREIGN KEY (department_id) REFERENCES departments(department_id)
);
```

Chapter 11: Indexes & Performance

Index Basics

An **index** is a data structure that speeds up data retrieval, like an index in a book.

How it works: Instead of scanning every row, the database uses the index to jump directly to matching data.

Trade-off:

- **Pro:** Faster SELECT queries
- **Con:** Slower INSERT, UPDATE, DELETE (index must be updated)

Types of Indexes

Single Column Index

CREATE INDEX idx_salary ON employees(salary);

Use when: Frequently filtering or sorting by that column.

Composite Index (Multiple Columns)

CREATE INDEX idx_dept_salary ON employees(department_id, salary);

Use when: Frequently filtering by multiple columns together.

Unique Index

CREATE UNIQUE INDEX idx_email ON employees(email);

Use when: Column must have unique values.

Full-Text Index (Text Search)

CREATE FULLTEXT INDEX idx_name ON employees(name);

SELECT * FROM employees WHERE MATCH(name) AGAINST('Rahul');

Use when: Searching large text fields.

When to Use Indexes

Good candidates for indexing:

- Columns used in WHERE clauses
- Columns used in JOIN conditions
- Columns used in ORDER BY
- Columns with many distinct values
- Columns in GROUP BY (sometimes)

Bad candidates:

- Columns with few distinct values (status, gender)
- Columns rarely used in queries
- Small tables (< 1000 rows)
- Columns with NULL values (sometimes)

Query Optimization Tips

1. Use Appropriate Indexes

```
-- SLOW - No index on salary
SELECT * FROM employees WHERE salary > 50000;

-- FAST - With index
CREATE INDEX idx_salary ON employees(salary);
SELECT * FROM employees WHERE salary > 50000;
```

2. Avoid SELECT *

```
-- SLOW - Retrieves all columns
SELECT * FROM employees;

-- FAST - Only needed columns
SELECT name, email, salary FROM employees;
```

3. Use Joins Instead of Subqueries (Usually)

```
-- SLOWER - Subquery
SELECT * FROM employees
WHERE department_id IN (SELECT department_id FROM departments WHERE region =
'North');

-- FASTER - Join
SELECT e.* FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE d.region = 'North';
```

4. Filter Early (WHERE before GROUP BY)

```
-- SLOWER - Aggregates all rows then filters
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;

-- FASTER - Filters first
SELECT department, AVG(salary)
FROM employees
WHERE salary > 40000
GROUP BY department
HAVING AVG(salary) > 50000;
```

5. Use LIMIT for Large Results

```
-- SLOW - Returns 1 million rows
SELECT * FROM transactions;

-- FAST - Returns first 100 rows
SELECT * FROM transactions LIMIT 100;
```

6. Avoid Functions on Indexed Columns

```
-- SLOW - Function prevents index use
SELECT * FROM employees WHERE YEAR(hire_date) = 2023;

-- FAST - Direct comparison
SELECT * FROM employees WHERE hire_date >= '2023-01-01' AND hire_date < '2024-01-01';
```

Analyzing Query Performance

EXPLAIN Statement

Shows how the database executes a query.

```
EXPLAIN SELECT * FROM employees WHERE salary > 50000;
```

Output shows:

- How many rows scanned
- Whether index is used
- Estimated cost
- Execution order

Example output:
```
Seq Scan on employees (cost=0.00..35.50 rows=500)
Filter: (salary > 50000)
```

EXPLAIN ANALYZE

Actually runs the query and shows real metrics.

```
EXPLAIN ANALYZE SELECT * FROM employees WHERE salary > 50000;
```

Chapter 12: Views, Procedures & Functions

Views

A **view** is a virtual table created from a SELECT query. It doesn't store data, just the query definition.

Create a View:

```
CREATE VIEW high_earners AS
SELECT name, salary, department_id
```

```
FROM employees
WHERE salary > 60000;
```

Use the View Like a Table:

```
SELECT * FROM high_earners;
SELECT name FROM high_earners WHERE department_id = 2;
```

Benefits:

- Simplify complex queries
- Restrict data access (show only certain columns)
- Consistency (single source of truth)

Drop a View:

```
DROP VIEW high_earners;
```

Stored Procedures

A **stored procedure** is a pre-compiled SQL code saved in the database and executed on demand.

Create a Procedure:

```
CREATE PROCEDURE get_employee_salary(emp_id INT)
AS
BEGIN
SELECT name, salary FROM employees WHERE employee_id = emp_id;
END;
```

Execute a Procedure:

```
EXEC get_employee_salary 1;
```

Benefits:

- Reusability
- Better performance (pre-compiled)
- Security (encapsulate logic)
- Reduced network traffic

Functions

Functions are similar to procedures but return a single value.

Create a Function (SQL Server):

```
CREATE FUNCTION CalculateBonus(@salary DECIMAL)
RETURNS DECIMAL
AS
BEGIN
DECLARE @bonus DECIMAL;
IF @salary > 60000
```

```
SET @bonus = @salary * 0.15;
ELSE
SET @bonus = @salary * 0.10;
RETURN @bonus;
END;
```

Use a Function:

```
SELECT name, salary, dbo.CalculateBonus(salary) AS bonus
FROM employees;
```

Triggers

A **trigger** is SQL code that automatically executes in response to specific events (INSERT, UPDATE, DELETE).

Example: Track salary changes

```
CREATE TRIGGER salary_audit
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
INSERT INTO salary_history (employee_id, old_salary, new_salary, changed_date)
VALUES (NEW.employee_id, OLD.salary, NEW.salary, NOW());
END;
```

Whenever an employee's salary is updated, this trigger automatically logs the change.

Chapter 13: Transactions & ACID Properties

COMMIT and ROLLBACK

Transaction - A sequence of SQL statements treated as one atomic unit.

COMMIT - Saves all changes permanently.

```
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 101;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 102;
COMMIT; -- Both updates saved
```

ROLLBACK - Undoes all changes since the transaction started.

```
BEGIN;
DELETE FROM employees WHERE id = 1;
-- Oops! Wrong employee
ROLLBACK; -- Deletion undone
```

SAVEPOINT

Create recovery points within a transaction. Can rollback to a specific savepoint instead of the entire transaction.

```
BEGIN;

UPDATE employees SET salary = salary * 1.10 WHERE department = 'IT';
SAVEPOINT after_it_raise;

UPDATE employees SET salary = salary * 1.05 WHERE department = 'HR';
-- Something went wrong with HR update
ROLLBACK TO after_it_raise; -- Only HR update is undone

COMMIT; -- Only IT raise is saved
```

ACID Properties

ACID ensures data reliability and consistency.

Atomicity

All-or-nothing principle. Either all statements in a transaction succeed or all are rolled back.

Example:

```
-- Money transfer: both must succeed or both fail
BEGIN;
UPDATE account1 SET balance = balance - 1000;
UPDATE account2 SET balance = balance + 1000;
COMMIT; -- Both happen, or neither happens
```

Consistency

Database moves from one valid state to another. All rules and constraints are maintained.

Example:

- Foreign keys are respected
- CHECK constraints are satisfied
- Primary key uniqueness maintained

Isolation

Concurrent transactions don't interfere. Each sees consistent data.

Isolation Levels:

- **READ UNCOMMITTED** - Dirty reads possible (rarely used)
- **READ COMMITTED** - Default, prevents dirty reads
- **REPEATABLE READ** - Prevents dirty and non-repeatable reads
- **SERIALIZABLE** - Highest isolation, like transactions run one-by-one

Durability

Once committed, data persists even if system fails.

Example:
Even if the server crashes immediately after COMMIT, the data is saved.

Chapter 14: SQL for Analytics & Reporting

Window Functions

Window functions perform calculations on a set of rows and return a value for each row.

Unlike aggregate functions that collapse rows, window functions preserve all rows.

ROW_NUMBER()

Assigns a unique sequential number to each row.

```
SELECT
name,
salary,
ROW_NUMBER() OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

Result:
name | salary | rank
Rahul | 70000 | 1
Amit | 65000 | 2
Priya | 60000 | 3

RANK()

Similar to ROW_NUMBER but handles ties.

```
SELECT
name,
salary,
RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

If two employees have same salary:
name | salary | rank
Rahul | 70000 | 1
Amit | 65000 | 2
Priya | 65000 | 2
Deepak | 60000 | 4

DENSE_RANK()

Similar to RANK but without gaps.

name | salary | rank
Rahul | 70000 | 1
Amit | 65000 | 2
Priya | 65000 | 2
Deepak | 60000 | 3 (not 4)

LAG() and LEAD()

Access previous or next row values.

```
SELECT
name,
salary,
LAG(salary) OVER (ORDER BY salary DESC) AS prev_salary,
LEAD(salary) OVER (ORDER BY salary DESC) AS next_salary
FROM employees;
```

Running Total

```
SELECT
name,
salary,
SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM employees;
```

Partition By

Divide rows into groups (partitions) and apply window functions separately to each.

```
SELECT
name,
department,
salary,
AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary,
salary - AVG(salary) OVER (PARTITION BY department) AS diff_from_avg
FROM employees;
```

This shows each employee's salary versus their department's average.

Use Cases in BI Tools

Year-over-Year Comparison

```
SELECT
month,
year,
revenue,
LAG(revenue) OVER (PARTITION BY month ORDER BY year) AS prev_year_revenue,
(revenue - LAG(revenue) OVER (PARTITION BY month ORDER BY year)) /
```

```
LAG(revenue) OVER (PARTITION BY month ORDER BY year) * 100 AS yoy_growth_percent
FROM monthly_sales;
```

Sales Ranking by Region

```
SELECT
region,
salesperson,
sales,
RANK() OVER (PARTITION BY region ORDER BY sales DESC) AS region_rank,
RANK() OVER (ORDER BY sales DESC) AS overall_rank
FROM sales_data;
```

Percentile Calculation

```
SELECT
name,
salary,
PERCENT_RANK() OVER (ORDER BY salary) AS percentile_rank
FROM employees;
```

Chapter 15: SQL Interview Questions & Scenarios

Common Interview Questions

Question 1: Second Highest Salary

Problem: Find the second highest salary.

Solution 1: Using OFFSET

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Solution 2: Using Subquery

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Solution 3: Using Window Function

```
SELECT salary
FROM (
SELECT salary, RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees
```

```
) ranked
WHERE rnk = 2;
```

Question 2: Find Duplicate Records

Problem: Find employees with duplicate names.

```
SELECT name, COUNT()
FROM employees
GROUP BY name
HAVING COUNT() > 1;
```

Question 3: Delete Duplicate Rows (Keep One)

Problem: Remove duplicates, keeping only the first occurrence.

```
DELETE FROM employees
WHERE employee_id NOT IN (
SELECT MIN(employee_id)
FROM employees
GROUP BY name
);
```

Question 4: Nth Highest Salary

Problem: Find the Nth highest salary (general solution).

```
SELECT salary
FROM (
SELECT DISTINCT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees
) ranked
WHERE rnk = N; -- Replace N with the value you want
```

Question 5: Employees with Above Average Salary

Problem: Find employees earning more than the average.

```
SELECT name, salary, department
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Question 6: Department with Highest Average Salary

Problem: Which department has the highest average salary?

```
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
ORDER BY avg_sal DESC
LIMIT 1;
```

Real-World SQL Scenarios

Scenario 1: Customer Lifetime Value

Problem: Calculate total spending per customer.

```
SELECT
customer_id,
customer_name,
COUNT(order_id) AS order_count,
SUM(order_amount) AS total_spent,
AVG(order_amount) AS avg_order_value
FROM orders
GROUP BY customer_id, customer_name
ORDER BY total_spent DESC;
```

Scenario 2: Monthly Growth Rate

Problem: Show month-on-month sales growth.

```
SELECT
month,
revenue,
LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
ROUND(((revenue - LAG(revenue) OVER (ORDER BY month)) /
LAG(revenue) OVER (ORDER BY month)) * 100, 2) AS growth_percent
FROM monthly_sales
ORDER BY month;
```

Scenario 3: Cohort Analysis

Problem: Track user retention by signup month.

```
WITH cohorts AS (
SELECT
DATE_TRUNC('month', signup_date) AS cohort_month,
user_id
FROM users
),
user_activity AS (
SELECT
DATE_TRUNC('month', activity_date) AS activity_month,
user_id
FROM user_activities
)
SELECT
c.cohort_month,
DATE_PART('month', ua.activity_month - c.cohort_month) AS months_since_signup,
COUNT(DISTINCT ua.user_id) AS active_users
FROM cohorts c
LEFT JOIN user_activity ua ON c.user_id = ua.user_id
```

```
GROUP BY c.cohort_month, months_since_signup
ORDER BY c.cohort_month, months_since_signup;
```

Scenario 4: Data Quality Check

Problem: Find data issues (NULL values, invalid entries).

```
SELECT
COUNT(*) AS total_rows,
SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END) AS missing_emails,
SUM(CASE WHEN salary <= 0 THEN 1 ELSE 0 END) AS invalid_salaries,
SUM(CASE WHEN hire_date > CURRENT_DATE THEN 1 ELSE 0 END) AS future_dates
FROM employees;
```

Scenario 5: Consecutive Days Active

Problem: Find users active on consecutive days.

```
WITH daily_activity AS (
SELECT DISTINCT user_id, activity_date
FROM user_logs
),
activity_with_gap AS (
SELECT
user_id,
activity_date,
ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY activity_date) AS seq,
activity_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY activity_date)
INTERVAL '1 day' AS grp
FROM daily_activity
)
SELECT
user_id,
MIN(activity_date) AS streak_start,
MAX(activity_date) AS streak_end,
COUNT() AS consecutive_days
FROM activity_with_gap
GROUP BY user_id, grp
HAVING COUNT() >= 3 -- At least 3 consecutive days
ORDER BY consecutive_days DESC;
```

Performance Interview Questions

Question: Optimize This Query

Original (Slow) Query:

```
SELECT * FROM orders
WHERE YEAR(order_date) = 2023;
```

Optimized Query:

```
-- Add index
CREATE INDEX idx_order_date ON orders(order_date);

-- Use direct date comparison instead of YEAR function
SELECT * FROM orders
WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01';
```

Why it's faster:

- Function on indexed column prevents index use
- Direct comparison allows index to work
- Fewer rows evaluated

Question: Which Query is More Efficient?

Query A: Subquery

```
SELECT e.name, e.salary
FROM employees e
WHERE e.department_id IN (
SELECT department_id FROM departments WHERE budget > 100000
);
```

Query B: Join

```
SELECT e.name, e.salary
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE d.budget > 100000;
```

Answer: Generally, Query B (Join) is more efficient because:

- Set-based operation
- Database optimizer handles it better
- Fewer intermediate result sets

SQL Quick Reference

Data Types

```
INT, BIGINT, FLOAT, DECIMAL
VARCHAR(n), CHAR(n), TEXT
DATE, TIMESTAMP, TIME
BOOLEAN
```

Essential Commands

```
SELECT ... FROM ... WHERE ... ORDER BY ... LIMIT
INSERT INTO ... VALUES
UPDATE ... SET ... WHERE
DELETE FROM ... WHERE
CREATE TABLE, ALTER TABLE, DROP TABLE
CREATE INDEX, CREATE VIEW
```

Operators

=, !=, >, <, >=, <=
AND, OR, NOT
IN, BETWEEN, LIKE
IS NULL, IS NOT NULL

Aggregate Functions

COUNT(), SUM(), AVG(), MIN(), MAX()
GROUP BY, HAVING

Join Types

INNER JOIN (only matches)
LEFT JOIN (all left + matches)
RIGHT JOIN (matches + all right)
FULL JOIN (all from both)
SELF JOIN (table to itself)

Advanced Features

Subqueries and CTEs (WITH clause)
Window functions (ROW_NUMBER, RANK, LAG, LEAD)
Transactions (BEGIN, COMMIT, ROLLBACK, SAVEPOINT)
Indexes (CREATE INDEX, UNIQUE INDEX)
Views (CREATE VIEW)

Key Takeaways

1. **SQL is about asking questions** - Think of what data you need, not how to code it
2. **Understand execution order** - JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
3. **Use appropriate tools** - JOINs for combining tables, CTEs for complex logic, subqueries for simple cases
4. **Optimize early** - Indexes, avoiding SELECT *, filtering early
5. **Practice joins and aggregations** - They're used in 80% of real queries
6. **Master window functions** - They're critical for analytics and BI
7. **Understand transactions** - ACID properties ensure data reliability
8. **Test and verify** - Always check results for correctness

SQL is a powerful language for data manipulation and analysis. Master these concepts and you'll be well-equipped for data engineering, analytics, and backend development roles.